View kısaca verinin görselleştirildiği katman.

.cshtml ile çalışır. Fakat bu dosya sadece Asp.Net'e özgüdür. Sadece Asp.Net tarafından render edilebilir.

C# ile HTML birleşmiş halidir. Render edildikten sonra çıktı olarak C# dosyaları görülemez. Bu teknolojiye Razor teknolojisi denir.

View oluşturma:

View'ler Views kalasörü altında bulunur ama zorunlu değil çünkü View() fonksiyonuna parametre olarak dosya yolu verilere istenilen View açılabilir. Ama Views dosyası altında tutmak aynı zaman da raconsaldır.

Action'a sağ tıklayıp add View seçeneği seçilip View oluşturulabilir.

View oluşturulunca, derleyici Action'ın bulunduğu Controller'ın isminde bir dosya oluşturuyor Views klasörü altında ve bu klasörün içerisinde View'i kayıt ediyor.

View oluştururken iki seçenek bulunur, Empty olan sadece View dosyası oluştururken Razor View yazan seçenek template seçenekleri sunar.

Bu Template'ler Create, Delete, Edit gibi bir çok işlem için hazır seçenekler sunar. Bu Templateler'den biri seçildiği takdirde bir model de talep ediyor.

View'e gönderme:

Model Bazlı Veri taşıma:

View() Fonksiyonu ile gönderilir. View fonksiyonun paramteresi ile gönderilir.

Bu yöntemde View'de modeli tanıtmak gerekmektedir.

```
@model List<EmptyToMvc.Models.Product>
```

Veriyi kullanabilmek için @Model kullanmak gerekmektedir. Yukarıda verilen tür ne ise @Model de o türde olacaktır.

Viewbag ile veri taşıma:

Viewbag View'e gönderilecek veriyi, dynamic bir şekilde tanımlanan bir değişkenle taşımamızı sağlayan bir yapıdır.

Viewbag'a hangi veri türü gelirse Viewbag o türde veriye dönüşür.

Viewbag Dynamic olarak çalıştığı için kod yazarken verilerin attribute'ları görünmeyebilir. Bu sebeple View içerisinde Viewbag'in türünü belirtmek gerekebilir. İhtiyaç değil keyfi.

```
<ul>
    @foreach (var product in ViewBag.products as List<EmptyToMvc.Models.Product>)
    {
        <li>@product.Name</li>
    }
</ul>
```

ViewData ile veri taşıma:

ViewData verilen veriyi Boxing ederek taşır. View'de Unboxing yapmak gerekir. Dynamic ile değil.

```
0 references
public IActionResult ViewDataBasedList()
{

    ViewData["products"] = products;
    #region ViewData ile Veri Gonderimi
    return View();
    #endregion

}
```

```
<ul>
    @foreach (var product in (List<EmptyToMvc.Models.Product>)ViewData["products"]) // Unboxing islemi
    {
        <li>@product.Name</li>
    }
</ul>
```

TempData ile veri taşıma:

ViewData'daki gibi Boxing ile veri taşır. Dynamic ile değil.

Aralarındaki fark, Aciton içerisinde bir Redirect işlemi varsa yani başka bir Action'a yönlendiriliyorsa diğerleri ile veri taşınamazken TempData ile veri taşınabilir.

Cookie'ler ile veri taşır.

Cookie üzerinden veriyi taşıdığı için ilgili verinin Cookie'ye dönüşmesi gerekiyor. Basit türler için sıkıntı olmuyor fakat Complex türler için(Nesne mesela) sterilize edilmesi gerek.

Bu sterilizasyon için JsonSerializer kütüphanesi kullanılıyor. JsonSerializer kütüphanesi içerisindeki Serialize() fonksiyonu ile yapılıyor. Örn

string data = JsonSerializer.Serialize(Product);

```
string data = JsonSerializer.Serialize(products);

TempData["products"] = data;
```

Veriyi TempData ile başka bir Action'a taşıdıktan sonra Desterilizasyon işlemi yapmak gerekir veriyi orijinal haline dönüştürmek için.

Bu işlem için yine JsonSerializer kütüphanesindeki Deserialize fonksiyonu kullanılabilir.

```
public IActionResult Index2()
{
    //var v1 = ViewBag.x;
    //var v2 = ViewData["x"];
    //var v3 = TempData["x"];

    var data= TempData["products"].ToString();
    JsonSerializer.Deserialize<List<Product>>(data);
```

Tuple ile View'e veri gönderme:

Tuple ile iki farklı veriyi aynı anda view'e gönderilebilir.

var userProduct = (product, user);

View tarafında ise model belirtmek için. Bir tuple nesnesi geleceğinden dolayı için (Birinci veri türü, İkinci veri türü)

Eğer yol olarak tanıtılmamışsa daha önce proje yolu ile vermek gerekir.

```
@model (P02_Viewler_View_Veri_Tasima.Models.Product, P02_Viewler_View_Veri_Tasima.Models.User)
```

Burada veri türünden sonra Tuple elemanlarına isim verilebilir.

```
@model (P02_Viewler_View_Veri_Tasima.Models.Product P, P02_Viewler_View_Veri_Tasima.Models.User U)
```

Aksi takdirde @Model ile erişirken item1 ve item2 olarak gözükecektir.

Razor View:

Asp.net Core MVC mimarisinde .cshtml uzantılı dosyalarda HTML ile birlikte yazılan, C# kodlarının server tarafında çalıştırılması sağlayan bir teknolojidir.

@ operatörü, razor operatörüdür.

Razor scope'ları ile değişken tanımlanabilir:

```
@*Değişken Tanımlama*@
@{
    string mesaj = "Merhaba Razor View!";
}
```

Razor ile değişken okuma :

```
@*Değişken okuma *@
@{
    int a = 5;
    Console.WriteLine(a);
}
```

Razor scope’ları tek bir scope gibi çalışır. Bir scope’da tanımlanan değer başka scope’da çağırılabilir:

```
@*Değişken okuma *@
@{
    int a = 5;
    Console.WriteLine(a);
}


@*Parçalı Scope Yapısı *@
@{
    Console.WriteLine(a); // Üst scope'da tanımlanan değişkene erişim sağlanabilir.
}
```

Razor içerisinde HTML kodları tanımlanabilir:

```
@*Razor içerisinde HTML kullanımı*@
@{
    <div>Merhaba</div>
}
```

Razor Tek satirlik işlemler yapılabilir:

```
@*Razor ile tek satırlık işlemler*@
@{
    <H3>@(5 + 3)</H3>
    <H3>@(33 % 5 == 3 ? "Evet" : "Hayir")</H3>
}
```

Burada yapılan işlemler parantez ile yapılmalıdır.

Kendi Konsepti olan yapılar:

```
@*Koşul ve dongu yapıları*@
@{
    int sayi = 5;
    if (sayi == 5)
    {
        <h3>evet</h3>
    }
    for (int i = 0; i < 5; i++)
    {
        <p>i'nin değeri: @i</p>
    }
}
@if (sayi == 5)
{
    <h3>evet</h3>
}
@for (int i = 0; i < 5; i++)
{
    <p>i'nin değeri: @i</p>
}
```